Pagesmith — The Editor Internals

A Developer Guide to `edit-mode.js`

David Falk

Abstract

This guide explains how Pagesmith's inline content editor (`edit-mode.js`) is *built*, for developers who maintain or extend it. We follow the file's own SURFACE MAP: how it boots and gates itself off production, the state model that survives the constant save → reload cycle, the `data-edit-*` binding protocol that lets parts become editable with zero editor code, the true-index trick that keeps list reordering correct, the Sections panel as a staging editor, the save/publish/revert backend, and the media upload pipeline. It closes with a recipe section for extending the editor and a catalogue of sharp edges to respect. Pagesmith is a generic, JSON-driven page builder; nothing here is specific to any one site.

Contents

1 Orientation: what edit-mode.js is

`edit-mode.js` is a single ∼2250-line IIFE that turns an ordinary live web page into an editable surface. It belongs to an engine/instance split: the *engine* (skeleton, catalogue of parts, and editor) is brand-neutral and renders whatever typed blocks are in a page's JSON; the *instance* is one config file (`site-config.js`, exposed as `window.SITE`) plus content JSON, CSS, and HTML shells.

The editor ships no site-specific strings. Everything it needs about *this* deployment — prod hostnames, wordmark, media-picker tabs, full-quality folders, friendly page names — arrives through `window.SITE`. Keep it that way: route anything site-shaped through config rather than hardcoding it.

The file opens with a SURFACE MAP comment that is the canonical table of contents for the code. Two gotchas, also called out at the top of the file, explain a great deal of what follows:

1. **The Sections panel is a staging editor.** Add / hide / delete / reorder are staged inside the panel and only written to the live page on *Apply & Save*. The on-page section count does not change as you click.

2. **Many "Done" buttons call `save(true)`, which reloads the page.** After most edits the page reloads and re-renders from the saved model. That is *why* unsaved inline edits live in `state.models`, not in the DOM — the DOM is disposable.

2 Boot & gating

2.1 Two gates, by design

`edit-mode.js` is not a static `<script>` tag. It is injected at runtime by `media-base.js`, and *only on non-prod hosts*:

```
// media-base.js
var isProdHost = !!(window.SITE && window.SITE.isProd(host));
// Load the inline editor runtime on dev/local hosts only (never on prod).
// edit-mode.js self-gates further: no-ops unless /.auth/me has the editor role.
if (!isProdHost) {
  var es = document.createElement('script');
  es.src = '/javascript/edit-mode.js';
  es.defer = true;
  (document.head || document.documentElement).appendChild(es);
}
```

So there are two independent gates (defence in depth), plus a third belt-and-braces check inside the file itself:

```
var host = (location.hostname || '').toLowerCase();
if (window.SITE && window.SITE.isProd(host)) return; // never on the live site
```

- **Host gate** (media-base): on a prod host the editor is never even downloaded — production users pay zero bytes for it.
- **Self-gate** (top of file): refuses to run if mistakenly placed on a prod page.
- **Role gate** (`boot`): even when loaded, does nothing unless the signed-in principal carries the `editor` role.

2.2 The role gate in `boot()`

```
function boot() {
  fetch('/.auth/me').then(r => r.ok ? r.json() : null).then(function (me) {
    var p = me && me.clientPrincipal;
    state.principal = p;
    var roles = (p && p.userRoles) || [];
    if (roles.indexOf('editor') === -1) {
      console.info('[engine-edit] not an editor - editor disabled.');
      return;                          // everything below never runs
    }
    /* build toolbar, wire events, expose window.EngineEdit */
  });
}
```

`/.auth/me` is the Azure Static Web Apps endpoint returning the current `clientPrincipal` (provider, user id, roles). Anonymous or non-editor users get a page indistinguishable from the public site.

Why fetch roles client-side if the server enforces them anyway? Because the client gate is purely cosmetic — it decides whether to *show* editing UI. Every mutating API call is independently enforced server-side via `requireEditor` (§9). A forged client check still bounces off the Functions.

2.3 The rest of boot

Past the gate, `boot()` does the housekeeping that lets the editor feel continuous across the constant reloads: it restores `state.editing` and scroll position from `sessionStorage`, builds the toolbar,

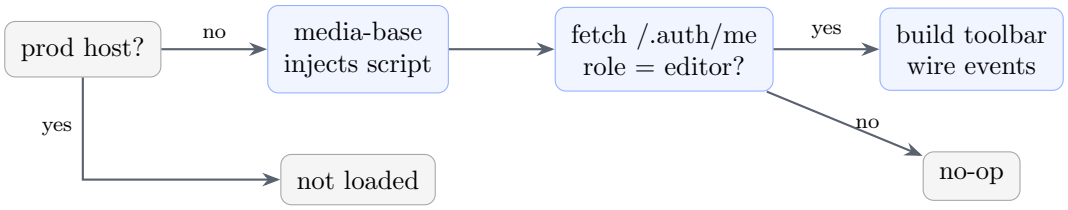

Figure 1: The three gates: host (no download), self-gate, and role.

wires the `engine-data-ready` and `engine-rebind` listeners, adds a `beforeunload` dirty-guard, and exposes `window.EngineEdit`. The `engine-data-ready` listener is the heart of the data flow (§3).

3 The state model

Everything the editor knows lives in one object:

```
var state = {
  file: null,      // primary page file key, e.g. 'page-music' (toolbar pill)
  data: null,      // primary page model - a POINTER into models[file]
  models: {},      // fileKey -> data   (the page file AND 'global')
  dirtyFiles: {},  // fileKey -> true   (which models have unsaved edits)
  editing: false,  // Edit vs Preview
  principal: null  // the SWA clientPrincipal from /.auth/me
};
```

3.1 Two models, and the `fileKeyOf` router

A rendered page is assembled from two JSON files: the **page file** (`index.json`, `page-music.json`, ...) holding the page's own blocks, and `global.json` holding the chrome shared by every page (nav, social, footer, questionnaire, tracking, theme). The header and footer are editable on every page but belong to `global`, not to the page you are on. So each editable node declares *which model it edits* via `data-edit-file`:

```
function fileKeyOf(node) {
  var a = node && node.closest && node.closest('[data-edit-file]');
  return (a && a.getAttribute('data-edit-file')) || state.file;
}
function modelOf(node) { return state.models[fileKeyOf(node)]; }
```

The rule: a node inside a `[data-edit-file="global"]` ancestor edits the global model; everything else defaults to the current page file. This is the most important routing decision in the editor — get it wrong and a header edit silently lands in the page model and is lost on the next page.

`state.data` and `state.file` are conveniences that track the current page; `state.models` is the source of truth.

3.2 Populating models: `onData`

The skeleton loads each JSON file and announces it; the editor folds it in:

```
window.__ENGINE_DATA__ = { file, data };
```

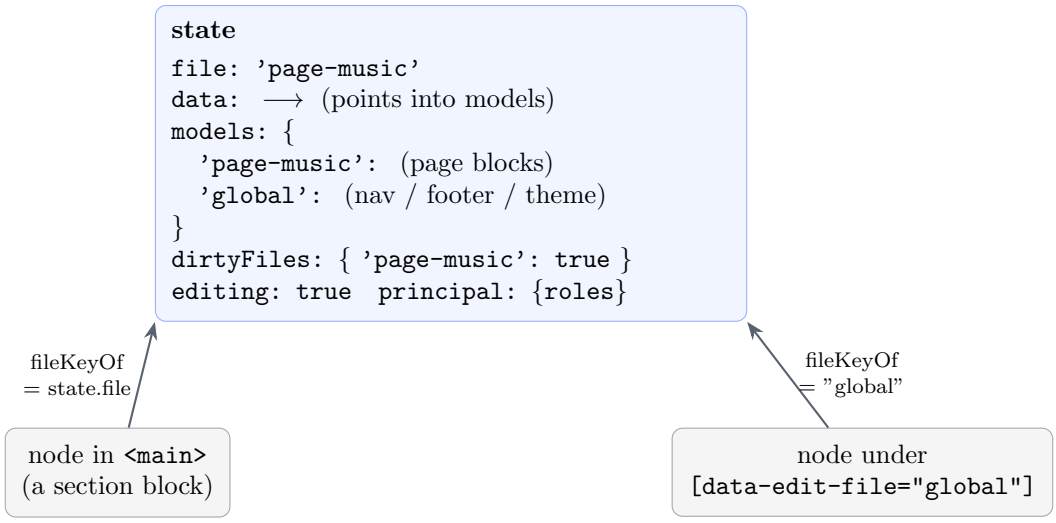

Figure 2: `fileKeyOf(node)` routes each node to its backing model.

```
document.dispatchEvent(new CustomEvent('engine-data-ready',
                                       { detail: { file, data } }));

function onData(file, data) {
  state.models[file] = data;
  if (file !== 'global') { state.file = file; state.data = data; }
  bindAll(); refreshToolbar();
  /* restore scroll after a save->reload if pendingScroll is set */
}
```

The `file !== 'global'` guard ensures the global model never clobbers the page pointers. Both models may arrive in either order, so `boot()` replays a global payload (`__ENGINE_GLOBAL__`) that raced ahead.

3.3 Dirty tracking

```
function isDirty() { return Object.keys(state.dirtyFiles).length > 0; }
function markDirty(fk) { if (fk) state.dirtyFiles[fk] = true; refreshToolbar(); }
```

Every mutation calls `markDirty(fileKeyOf(node))`; the toolbar reflects it. `save()` clears `dirtyFiles` only after the POSTs succeed.

4 The data-edit-* binding protocol

The editor does not know your blocks' data shapes. Parts *tag* their DOM with `data-edit-*` attributes, and the editor wires behaviour onto whatever it finds. This is the entire contract between a part and the editor.

Attribute	Behaviour
`data-edit-text="path"`	inline plain-text (`contentEditable`); writes `textContent`
`data-edit-html="path"`	inline rich text; writes `innerHTML`
`data-edit-image="path"`	click → media picker / section editor
`data-edit-attr="path|attr"`	click → `prompt()` edits one attribute
`data-edit-bg="blob"`	click → replace a fixed CSS background blob
`data-edit-list="path"`	drag-reorder + the list panel
`data-edit-file="key"`	routes all of the above to a model
`data-item-index="i"`	on each LIST CHILD, its true array index

4.1 `bindAll()` — idempotent, after every render

```
function bindAll() {
  if (!Object.keys(state.models).length) return;
  document.querySelectorAll('[data-edit-text]').forEach(n => bindText(n, false));
  document.querySelectorAll('[data-edit-html]').forEach(n => bindText(n, true));
  document.querySelectorAll('[data-edit-image]').forEach(bindImage);
  document.querySelectorAll('[data-edit-attr]').forEach(bindAttr);
  document.querySelectorAll('[data-edit-bg]').forEach(bindBg);
  document.querySelectorAll('[data-edit-list]').forEach(bindList);
  setEditing(state.editing);
}
```

`bindAll()` runs after each (re)render — from `onData` and from the `engine-rebind` event a part fires after re-rendering itself. It must be idempotent, so each binder guards with a per-node flag:

```
function bindText(n, isHtml) {
  if (n.__engineBound) return; n.__engineBound = true; // bind once
  /* ... */
}
```

This is why `bindAll()` can be called freely after any partial re-render without stacking duplicate listeners. When you add a new binder (§10), keep the `__engineBound` guard — it is load-bearing.

4.2 Paths: tokenize then resolve

The glue between a node and the model is a path string like `sections[2].items[0].value`. Two functions do all the addressing:

```
function tokenize(path) {
  return String(path)
    .replace(/\[(\d+)\]/g, '.$1')   // sections[2] -> sections.2
    .split('.')
    .filter(s => s !== '');
}
function getByPath(obj, path) {
  var t = tokenize(path), o = obj;
  for (var i = 0; i < t.length; i++) { if (o == null) return undefined; o = o[t[i]]; }
  return o;
}
function setByPath(obj, path, val) {
  var t = tokenize(path), o = obj;
```

```
  for (var i = 0; i < t.length - 1; i++) {
    var k = t[i];
    // create the missing container: array if the NEXT token is numeric, else object
    if (o[k] == null || typeof o[k] !== 'object')
      o[k] = /^\d+$/.test(t[i + 1]) ? [] : {};
    o = o[k];
  }
  o[t[t.length - 1]] = val;
}
```

Worked example. `sections[2].items[0].value` tokenizes to `['sections','2','items','0','value']`. Against:

```
{
  "sections": [
    { "type": "heading", "text": "Hi" },
    { "type": "text", "body": "..." },
    { "type": "links", "items": [ { "value": "https://a", "label": "A" } ] }
  ]
}
```

`getByPath` walks `model` → `sections` → `[2]` → `items` → `[0]` → `value` and returns `"https://a"`. `setByPath` walks the same route but stops one short, then assigns. Its auto-vivify rule (array when the next token is numeric, else object) lets a part bind to a path that does not exist yet — the first edit materialises the container correctly.

4.3 Text/HTML, image, attr, bg

Text binding writes through on `input`:

```
n.addEventListener('input', function () {
  var m = modelOf(n); if (!m) return;
  setByPath(m, n.getAttribute(isHtml ? 'data-edit-html' : 'data-edit-text'),
            isHtml ? n.innerHTML : n.textContent);
  markDirty(fileKeyOf(n));
});
```

Plain text intercepts Enter to blur; rich text stores raw `innerHTML` (a sharp edge, §11). **image** clicks open the section's unified Edit panel (or, for legacy standalone media, a floating Replace/Remove toolbar); `replaceMedia` opens the picker, writes the chosen path with `setByPath`, repaints in place, and marks dirty. **attr** splits `path|attr`, prompts, and writes both model and live attribute. **bg** names a fixed blob and replaces its bytes at that exact filename.

5 List reordering: the data-item-index contract

On-page drag-to-reorder hides a subtle, expensive bug class, and the editor avoids it deliberately.

5.1 The bug class

A list block renders DOM children from a backing array — but a part may render *fewer* children than the array has items (it might skip a blank URL or an invalid image). Now array index and DOM position have desynced. If reorder spliced by DOM position, it would move the *wrong* array

element whenever an earlier item was filtered out: the visible order changes, but a different, invisible item actually moved. Silent corruption.

5.2 The fix: stamp the true index

Each list part stamps every rendered child with its true array index via `data-item-index` (documented in the `blocks.js` editing protocol):

```
// blocks.js - a gallery part stamping the real array index j on each child
var img = ctx.el('img', { src: ctx.mediaUrl(src), alt: (im && im.alt) || '',
                          'data-item-index': j /* TRUE index */ });
```

The editor prefers the stamp over DOM position, falling back only when absent:

```
function itemIndexOf(container, child) {
  if (child && child.getAttribute) {
    var raw = child.getAttribute('data-item-index');
    if (raw != null && raw !== '') { var n = parseInt(raw, 10); if (!isNaN(n)) return n;
    }
  }
  return Array.prototype.indexOf.call(container.children, child); // fallback
}
```

The DOM-position fallback exists for simple one-child-per-item lists, but **any part that filters items must stamp**, or it reintroduces the desync. That is the rule when authoring a part.

5.3 The splice

`startReorder` and `dragover` record *true* indices; `endReorder` splices the model and saves:

```
function endReorder() {
  var d = dragNow; dragNow = null;
  if (d && d.to != null && d.to !== d.from) {
    var fk = fileKeyOf(d.container), model = state.models[fk];
    var arr = getByPath(model, d.path);
    if (Array.isArray(arr) && d.from < arr.length) {
      arr.splice(d.to, 0, arr.splice(d.from, 1)[0]); // move from -> to
      markDirty(fk);
      save(true);                                    // persist + reload
    }
  }
}
```

Because `from` and `to` are true array indices, the splice is correct even when DOM and array disagree on length.

6 The Sections panel as a staging editor

`openSectionsNav()` edits page structure — reorder, hide/show, delete, jump, edit. The crucial design choice: it edits a *working copy*, not the live page. Nothing changes the on-page section count until *Apply & Save*.

6.1 The working list

It assembles one flat `working[]` from two sources, in render order:

```
var working = [];
// 1) built-in [data-section] nodes from the shell (hand-built static sections)
builtinEls.forEach(s => working.push({ kind: 'builtin', key, label, el: s, hidden }));
// 2) catalogue blocks from state.data.sections[]
blocks.forEach(b => working.push({ kind: 'block', block: b, label,
                                   hidden: b.visible === false, deleted: false }));
```

Hide, Show, Delete, Undo, and drag-reorder mutate `working[]` and re-render the panel rows. The live page is untouched.

6.2 `applySave` — splitting built-ins from blocks

On Apply & Save, the working list is split back into the structures the renderer honours:

```
function applySave() {
  var builtinOrder = working.filter(w => w.kind === 'builtin');
  // built-in ordering -> state.data.order (array of [data-section] keys)
  if (builtinOrder.length) state.data.order = builtinOrder.map(w => w.key);
  else delete state.data.order;
  // built-in visibility -> state.data.hiddenSections
  var hk = builtinOrder.filter(w => w.hidden).map(w => w.key);
  if (hk.length) state.data.hiddenSections = hk; else delete state.data.hiddenSections;
  // catalogue blocks -> state.data.sections (minus deleted; visible:false for hidden)
  var newBlocks = working.filter(w => w.kind === 'block' && !w.deleted).map(function (w)
    {
    if (w.hidden) w.block.visible = false; else if ('visible' in w.block) delete w.block.
    visible;
    return w.block;
  });
  if (newBlocks.length || hadBlocks) state.data.sections = newBlocks;
  markDirty(state.file);
  overlay.remove();
  save(true);   // persist + reload; page re-renders from the saved model
}
```

Three structures carry the result: `state.data.order` (built-in order), `state.data.hiddenSections` (built-ins to hide), and `state.data.sections` (catalogue blocks, ordered, hidden ones flagged, deleted ones dropped).

6.3 Why the count is unchanged live — and a real limitation

The change lands in the model only on Apply, and the page re-renders only on the subsequent reload, so the live `#engine-sections` count stays put while you fiddle. This is the staging contract — and why tests that assert against the live DOM mid-edit get false failures. Assert after the save → reload, or against `state.data.sections`.

A genuine limitation: `sections.js` appends the `#engine-sections` host (all catalogue blocks) to `<main>` *after* every built-in `[data-section]`. `applySave` splits the two into separate structures, so the relative order of built-ins vs blocks is unrepresentable — dragging a block above a built-in has no on-page effect.

7 Save, and why edits live in the model

```
function save(reloadAfter) {
  var files = Object.keys(state.dirtyFiles);
  if (!files.length) {
    if (reloadAfter) { stashScroll(); setTimeout(location.reload, 300); }
    return;
  }
  freezeQuestionnaireNames();
  Promise.all(files.map(fk =>
    fetchJson('/api/save-content/' + fk, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(state.models[fk])
    })
  )).then(function () {
    state.dirtyFiles = {}; refreshToolbar();
    if (reloadAfter) { stashScroll(); setTimeout(location.reload, 600); }
  }).catch(() => toast("Couldn't save your draft - check your connection."));
}
```

`save()` POSTs only the dirty models, each to `/api/save-content/<fileKey>`, which writes that one JSON file into the dev (draft) container. Page and global go up in parallel.

7.1 Why edits live in `state.models`, not the DOM

This is the central architectural fact. After almost any structural edit the page reloads and re-renders from the saved JSON; the DOM you were editing is thrown away. Therefore every inline edit writes through to `state.models` immediately — the DOM change is just a live preview — and a reorder/add/delete mutates the model then `save(true)`. When the model is the source of truth, a re-render can never lose an edit already written to it. A feature that mutates only the DOM will vanish on the next reload; do not write one.

7.2 The reload gotcha (especially for tests)

`save(true)` reloads after ∼300–600 ms, stashing `scrollY` so `onData` can restore it. The reload is what re-renders the page from the saved model. For tests this is a classic trap: an assertion firing immediately after an action that triggers `save(true)` reads the pre-reload DOM and sees stale content. Wait for the reload to settle, or assert against `window.EngineEdit.state.models`.

8 Media picker & upload pipeline

`openMediaPicker(prefix, onChoose, opts)` shows a tabbed browser. The tabs come straight from config:

```
var MEDIA_TABS = (window.SITE && window.SITE.mediaTabs) || [];
```

Each tab is one media folder, listed via `/api/media?prefix=...`. Picking an existing tile calls `onChoose(path)`; uploading routes through `doUpload`. `opts.replaceName` switches the picker into "replace a fixed blob" mode (used for CSS backgrounds): it copies the bytes to a known filename and fires `opts.onReplaced`.

8.1 Client optimisation, then upload

Before upload, `prepareMedia` optimises in the browser: images (`jpg/png/webp`) are downscaled to a 2560 px max edge and re-encoded to WebP at quality 0.82 (kept only if smaller); videos are size-capped at 50 MB (big ones steered to YouTube); everything is bounded by a ∼90 MB transport ceiling. **Full-quality folders are exempt**: if the target prefix matches `SITE.isFullQuality(prefix)`, the file passes through untouched. The optimised blob is base64-encoded and POSTed as JSON to `/api/media-upload` (base64-JSON keeps the Function dependency-free — no multipart parser).

8.2 Server side: WebP + variants, with a mirrored rule

`/api/media-upload` re-runs the same policy as a backstop and generates responsive variants:

```
// api/media-upload - the full-quality rule MUST mirror SITE.isFullQuality
const FULL_QUALITY_PREFIXES = (process.env.MEDIA_FULL_QUALITY_PREFIXES || '')
  .split(',').map(s => s.trim()).filter(Boolean);
const isFull = isFullQuality(blobName);
if (!isFull && WEBP_FROM.includes(ext))      { /* toWebp(buffer) */ }
else if (!isFull && VIDEO_EXT.includes(ext)) { /* transcodeToWebMp4 */ }
// then generateResponsiveVariants(...) for non-full-quality images
```

The full-quality bypass must match on both sides: the client reads `SITE.fullQualityMedia`; the server reads the `MEDIA_FULL_QUALITY_PREFIXES` app setting. If they drift you get the worst of both worlds. Keep them in sync.

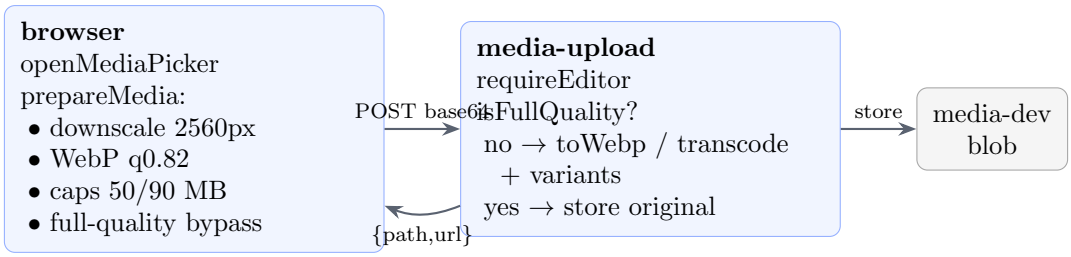

Figure 3: The upload pipeline; the full-quality rule is mirrored on both ends.

9 Publish / Revert and the backend

9.1 Draft vs published

Two container sets: **draft** (`content-dev`, `media-dev`) — what the editor reads and writes — and **published** (`content-prod`, `media-prod`) — what the live site serves. The editor only ever writes to draft. Publishing and reverting are server-side copies between the sets.

9.2 Publish → /api/promote

`publish()` warns if there are unsaved edits (Publish ships the last *saved* draft, not in-memory edits), confirms, then POSTs `/api/promote`, which copies every content JSON dev → prod *skipping pages marked* `visibility:'dev'` in `global.json` (Dev-only pages never go live), then syncs media dev → prod via server-side blob copy from the public-read draft URL (no bytes flow through the Function).

9.3 Revert → /api/revert

The inverse: copy every content file prod → dev, restoring the draft to the last published version, then reload.

9.4 Auth behind the SWA gate

Every mutating endpoint calls `requireEditor`, reading the `x-ms-client-principal` header that Static Web Apps injects (base64 JSON with roles) and requiring `editor`:

```
// api/_lib/auth.js
function requireEditor(context, req) {
  if (isEditor(req)) return true;   // checks x-ms-client-principal roles
  context.res = { status: 403, body: { error: 'Not authorized - editor role required' }
    };
  return false;
}
// EDITOR_AUTH_DISABLED=true bypasses this locally (never set in cloud).
```

The browser never holds a storage key; the Functions reach blob storage via scoped per-container SAS tokens in app settings.

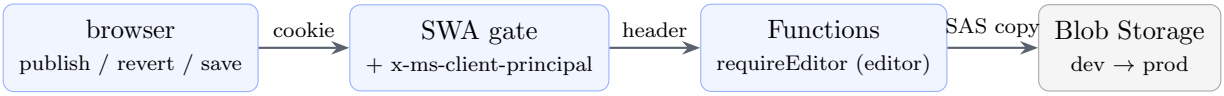

Figure 4: Request flow: browser → SWA → Functions → blob.

10 How to extend the editor

10.1 Make a part editable

You write zero editor code. In your part's render (`blocks.js` / `site-blocks.js`), tag the DOM:

```
// inside a part's render(block, ctx)
ctx.el('h2', { 'data-edit-file': ctx.file,
               'data-edit-text': ctx.path + '.title', text: block.title });
ctx.el('img', { 'data-edit-file': ctx.file,
                'data-edit-image': ctx.path + '.src', src: ctx.mediaUrl(block.src) });
var grid = ctx.el('div', { 'data-edit-file': ctx.file,
                           'data-edit-list': ctx.path + '.items',
                           'data-edit-list-label': 'links' });
block.items.forEach(function (it, j) {
  grid.appendChild(ctx.el('a', { 'data-item-index': j, href: it.url, text: it.label }));
    // stamp j!
});
```

`bindAll()` discovers and wires these on the next render. Stamp `data-item-index` on filtered lists (§5) and set `data-edit-file` (§3).

10.2 Add a new inspector field type

Fields are dispatched by a `type` string (`text`, `textarea`, `url`, `image`, `bool`, `select`, `csv`, `color`, `embed`). To add one, extend the field-renderer switch with a new `case`: build the input, seed it from the current value via `getByPath`, and on change call the supplied `onChange(value)` callback. The case must not write the model directly — the callback owns `setByPath` + `markDirty` + the live re-render. Use the `color` case as a template; it is the most recently added type and shows the full pattern.

10.3 Add a panel to the More menu

Toolbar buttons are built in `buildToolbar()`; secondary tools live in the `#engine-more` dropdown:

```
var btnThing = el('button', { text: 'My Thing', title: '...', on: { click: openThingPanel
    } });
[btnTheme, btnBg, /* ... */, btnThing].forEach(x => moreMenu.appendChild(x));
```

Write `openThingPanel()` following the slide-in convention: an `.engine-panel` with an `.engine-phead` (title + `helpBtn('thing')` + Close) and an `.engine-pbody`, then `panelize()` it into an `.engine-overlay`. Persist with `markDirty(state.file)` + `save()`; if Done should re-render the page, call `save(true)`.

10.4 The per-tool HELP registry

Each panel shows a round "?" via `helpBtn(key)`, opening `openHelpTopic(key)` against the `HELP` registry:

```
var HELP = {
  thing: {
    title: 'My Thing',
    intro: 'One sentence on what this does.',
    body: [
      ['A subheading', ['A bullet.', 'Another - note the empty-field behaviour.']]
    ]
  }
  /* sections, pages, theme, style, editsection, lists, questionnaire,
     tracking, background, media, addsection ... */
};
```

When you add a panel, add a matching `HELP` entry and wire `helpBtn('thing')` into its header. Existing entries are deliberately plain-language and call out the "what if I leave this empty?" edge cases — match that tone.

11 Known sharp edges (respect these)

These are real, current behaviours — things to keep in mind, not to "fix" in passing into a regression.

1. **Unsaved page edits vs Pages-panel navigation.** Inline edits write to `state.models[pageFile]` and set `dirtyFiles`, but are only POSTed by `save()`. Some navigation flows (Pages "Open", create-page, "Add to menu") run `saveGlobal` (persisting only `global`) then `location.href` away — the dirty *page* model is never saved. The `beforeunload` guard shows the browser's generic prompt, but it is easy to dismiss. New navigation flows should save the page model first (or warn).

2. **The staging / live split.** The Sections panel stages changes; the page reflects them only after Apply & Save → reload. Do not assert against the live count mid-edit, and do not add a "live as you click" feature without reconciling it with `applySave`'s built-in/block split.

3. **The save → reload timing.** `save(true)` reloads ∼300–600 ms later. Anything reading the DOM right after triggering it sees stale content. Wait for the reload, or read `state.models`.

4. `data-edit-html` **stores raw contentEditable HTML.** No sanitisation; browser-injected `<div>`/`<br>` and pasted markup are persisted. Acceptable for trusted editors; mind it if you widen who can edit.

5. **Built-in vs block ordering is not unified** (§6).

6. **Background replace can show a cached old image.** Fixed-blob backgrounds reuse the filename; `applyLiveBackground` cache-busts the element currently showing it, but a stylesheet-rule background can't be repainted and the user is told to hard-refresh. Expected, not a bug.

7. **The full-quality rule lives in two places** — client `SITE.fullQualityMedia` and server `MEDIA_FULL_QUALITY_PREFIXES` must agree.